In [ ]:
```
import marimo as mo
```

3.(a) For each molecule in the MUTAG dataset, compute the number of 6-membered rings as a new annotation. You may use an automated/manual approach.

(b) Using these ring counts as target labels, train a GNN to predict the number of 6-membered rings in a molecule. You may treat this as a regression or a multi-class

classification problem; justify your choice. Report appropriate evaluation metrics on a held-out test set.

In [ ]:
```
import torch
import torch.nn.functional as F
import networkx as nx
from torch.nn import Linear
from torch_geometric.datasets import TUDataset
from torch_geometric.utils import to_networkx
from torch_geometric.loader import DataLoader
from torch_geometric.nn import GCNConv, global_mean_pool
```

In [ ]:
```
# 1.Data Preparation and annotate
def count_6_membered_rings(data) -> torch.Tensor:
    G = to_networkx(data, to_undirected=True)
    cycles = nx.cycle_basis(G)
    count = sum(1 for cycle in cycles if len(cycle) == 6)
    return torch.tensor([count], dtype=torch.float)
```

In [ ]:
```
# Load the raw MUTAG dataset
dataset = TUDataset(root='data/TUDataset', name='MUTAG')

# Annotate every graph with its 6-membered ring count
annotated_data = []
for data in dataset:
    data.y = count_6_membered_rings(data)   #overwrite label with ring count
    annotated_data.append(data)

#Print a quick summary of the annotation
counts = [int(d.y.item()) for d in annotated_data]
print(f"Dataset size      : {len(annotated_data)}")
print(f"Ring-count values : min={min(counts)}, max={max(counts)}, "
      f"mean={sum(counts)/len(counts):.2f}")
print(f"Distribution      : { {v: counts.count(v) for v in sorted(set(counts
print()
```

```
Dataset size      : 188
Ring-count values : min=0, max=6, mean=2.13
Distribution      : {0: 2, 1: 48, 2: 82, 3: 38, 4: 16, 5: 1, 6: 1}
```

In [ ]:
```
# Train / test split  (80 % / 20 %, reproducible)
torch.manual_seed(42)
n_total  = len(annotated_data)
```

```
n_train  = int(0.8 * n_total)
n_test   = n_total - n_train

# Use index-based split so DataLoader receives proper Data objects
indices      = torch.randperm(n_total).tolist()
train_data   = [annotated_data[i] for i in indices[:n_train]]
test_data    = [annotated_data[i] for i in indices[n_train:]]

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader  = DataLoader(test_data,  batch_size=32, shuffle=False)
```

In [ ]:
```
# GNN Model
class RingPredictorGNN(torch.nn.Module):
    def __init__(self, num_node_features: int):
        super().__init__()
        self.conv1 = GCNConv(num_node_features, 32)
        self.conv2 = GCNConv(32, 32)
        self.conv3 = GCNConv(32, 32)
        self.lin   = Linear(32, 1)   # single output → regression
    def forward(self, x, edge_index, batch):
        # Message-passing layers
        x = F.relu(self.conv1(x, edge_index))
        x = F.relu(self.conv2(x, edge_index))
        x = self.conv3(x, edge_index)           # no activation before pooli
        # Graph-level outpt
        x = global_mean_pool(x, batch)

        # Regression head
        return self.lin(x)
model=RingPredictorGNN(num_node_features=dataset.num_node_features)
optimizer=torch.optim.Adam(model.parameters(), lr=0.001)
criterion=torch.nn.MSELoss()   # MSE: penalises large deviations from true c
```

In [ ]:
```
# Training + evaluation pipeline
def train() -> float:
    #One epoch of training. Returns mean MSE loss over training graphs.
    model.train()
    total_loss = 0.0
    for batch in train_loader:
        optimizer.zero_grad()
        out  = model(batch.x, batch.edge_index, batch.batch)   # [N, 1]
        loss = criterion(out, batch.y.view(-1, 1))
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * batch.num_graphs
    return total_loss / len(train_loader.dataset)


def evaluate(loader) -> tuple[float, float, float]:
    #Evaluate the model on a DataLoader.
    #Returns (MSE, MAE, R²) averaged over all graphs.
    model.eval()
    total_mse = 0.0
    total_mae = 0.0
    all_preds, all_targets = [], []
```

```
    with torch.no_grad():
        for batch in loader:
            out = model(batch.x, batch.edge_index, batch.batch)
            y   = batch.y.view(-1, 1)
            total_mse += F.mse_loss(out, y).item() * batch.num_graphs
            total_mae += F.l1_loss(out, y).item()  * batch.num_graphs
            all_preds.append(out)
            all_targets.append(y)

    n        = len(loader.dataset)
    mse      = total_mse / n
    mae      = total_mae / n

    # R² = 1 - SS_res / SS_tot
    preds   = torch.cat(all_preds)
    targets = torch.cat(all_targets)
    ss_res  = ((targets - preds) ** 2).sum().item()
    ss_tot  = ((targets - targets.mean()) ** 2).sum().item()
    r2      = 1.0 - ss_res / (ss_tot + 1e-8)

    return mse, mae, r2
```

In [ ]:

```
# Training loop
NUM_EPOCHS = 100
REPORT_EVERY = 20

print("Starting Training…")
print(f"{'Epoch':>6}  {'Train MSE':>10}  {'Test MSE':>9}  {'Test MAE':>9}  {
print("—" * 55)

for epoch in range(1, NUM_EPOCHS + 1):
    train_mse = train()
    if epoch % REPORT_EVERY == 0:
        test_mse, test_mae, test_r2 = evaluate(test_loader)
        print(f"{epoch:>6}  {train_mse:>10.4f}  {test_mse:>9.4f}  "
              f"{test_mae:>9.4f}  {test_r2:>8.4f}")


# final evaluation
test_mse, test_mae, test_r2 = evaluate(test_loader)
print()
print("Final Test-Set Metrics")
print("—" * 55)
print(f"  MSE  (Mean Squared Error)      : {test_mse:.4f}")
print(f"  MAE  (Mean Absolute Error)     : {test_mae:.4f}")
print(f"  RMSE (Root Mean Squared Error) : {test_mse**0.5:.4f}")
print(f"  R²   (Coefficient of Det.)     : {test_r2:.4f}")
```

```
Starting Training…
 Epoch   Train MSE   Test MSE   Test MAE   Test R²
──────────────────────────────────────────────────
    20      0.8779     0.8341     0.7184    0.0649
    40      0.6951     0.5694     0.6129    0.3617
    60      0.6268     0.4668     0.5589    0.4766
    80      0.6185     0.4797     0.5716    0.4622
   100      0.6108     0.4752     0.5725    0.4672

Final Test-Set Metrics
──────────────────────────────────────────────────────────
  MSE  (Mean Squared Error)      : 0.4752
  MAE  (Mean Absolute Error)     : 0.5725
  RMSE (Root Mean Squared Error) : 0.6894
  R²   (Coefficient of Det.)     : 0.4672
```

a) I used a cycle basis function for each molecule in MUTAG, by converting PyTorch Geometric graph to a NetworkX graph, extracted its cycle basis, and counted only cycles of length exactly 6. This count replaced the original MUTAG label.

b)

- I chose regression over classification for three reasons: the targets metric distances between labels are meaningful, treating them as unordered classes discards that structure, and a single output neuron generalises to unseen counts without retraining.
- A 3-layer GCN (GCNConv) with hidden dimension 32, followed by global mean pooling to produce a graph-level embedding, then a single linear layer outputting one scalar.
- Adam optimizer with MSE loss over 100 epochs, 80/20 train-test split.

An R^2 of ~0.48 means the model explains roughly half the variance in ring counts — reasonable for a compact 3-layer GCN on a small dataset(188 molecules).